

Programación Concurrente en Java

Curso 2006-2007



¿Qué es la Programación Concurrente?

- Diseño basado en varias actividades independientes
 - Conceptualmente se ejecutan en paralelo
 - En un nodo, multiplexa el tiempo de procesador entre las tareas
 - En un sistema distribuido, paralelismo real
- Las actividades deben cooperar entre sí
 - Comunicación
 - Sincronización
- Actividades = procesos o threads (hilos)

Utilidad de la programación concurrente

- Mejora las características de:
 - Flexibilidad
 - Interactividad
 - Eficiencia
- Facilita el diseño de sistemas
 - Reactivos (responden a estímulos externos)
 - Con actividades lógicamente separadas y distintos niveles de prioridad
 - Con actividades periódicas
 - Ej.- atención a varios clientes, timeouts, etc.

Dificultades a resolver

- Puede darse cualquier intercalado de ejecución
- Los programas deben ser correctos bajo *cualquier intercalado posible*
 - Ejemplo.- acceso a cuenta bancaria
- Posibles interferencias entre tareas
 - Resultado incorrecto bajo ciertos intercalados (no determinismo)
- Vivacidad, interbloqueos

Proceso

- Abstracción proporcionada por el SO (ej Unix)
 - El SO planifica y ejecuta varios procesos
 - Algoritmo de planificación
 - Cooperativo (cada proceso se autolimita)
 - Expulsivo (el sistema limita el tiempo dedicado a cada proceso)
 - Cada proceso mantiene su propio estado
 - Pila
 - Áreas de datos
 - Ficheros abiertos, etc.
 - Existe una jerarquía de objetos (relación padre-hijo)
 - Padre e hijo comparten información
 - Ficheros abiertos
 - Pipes y streams
 - Sockets
 - Pueden comunicarse/sincronizarse

Thread (Thread=hilo=proceso ligero)

- Hilos = Actividades concurrentes dentro de un proceso
- Comparten el contexto del proceso padre
 - Variables compartidas
 - Ficheros abiertos, etc.
- Pero cada hilo mantiene una parte local
 - Pila local
 - Variables locales, etc
- Creación/destrucción/comunicación/cambio estado mucho más eficiente que con procesos
- La memoria compartida facilita la comunicación y sincronización
 - Pero aparecen posibles interferencias
- Nos centramos en hilos (Threads)
 - Asumimos acceso a variables comunes

Interferencias

- Muchas operaciones suponen tanto lecturas como escrituras
 - Ej.- `i++` consiste a bajo nivel en varias operaciones primitivas
 - `r<-[i]`, `inc r`, `[i]<-r`
- Esas operaciones no se tratan como atómicas
 - El planificador puede interrumpirla en cualquier punto, cediendo el control a otro hilo
 - Puede darse cualquier intercalado de ejecución
- Ej dos hilos A, B ejecutan concurrentemente sendas instrucciones `i++`
 - A ejecuta `r<-[i]`
 - B ejecuta `i++` (todos los pasos)
 - A ejecuta `inc r`, `[i]<-r`
 - Hemos perdido el cambio introducido por B
- Solución
 - Concepto de sección crítica (exclusión mútua)
 - Sincronización (reservas, semáforos, monitores, etc.)

¿Porqué Java?

- Incorpora construcciones para Programación Concurrente
 - Los hilos forman parte del modelo del lenguaje
 - Aplicación = hilos usuario + hilos sistema (ej.- para GC y GUI)
 - Hilos usuario.- uno para el método main(), y podemos crear otros
 - La aplicación termina cuando finalizan todos sus hilos usuario
 - Facilita el desarrollo de aplicaciones concurrentes
- Lenguaje conocido (Prog. Avanzada)
- Demandado por el mercado
- Además
 - Facilidades para programación en red
 - Plataforma de distribución de bajo nivel
 - Facilidades para Objetos Distribuidos
 - Plataforma de distribución de nivel intermedio

Prog. Concurrente en Java

- Creación de hilos
- Ej.- Servidor multi-hilo
- Ciclo de vida de los hilos
- Estados de un hilo vivo
- Sincronización
 - Exclusión mútua
 - Espera y notificación
- Ejemplo
- Interbloqueos

Creación de hilos

- Crear nueva clase como extensión de **Thread**
`class X extends Thread {`
 ...
 `public void run() {..} // codigo del hilo`
}
- ... `X h= new X(); h.start();`
- Implementar interface **Runnable**
`class X implements Runnable {`
 `Public void run() {..} // codigo del hilo`
}
- .. `Thread h= new Thread(new X()); h.start();`
- La actividad se inicia al aplicar el método **start** sobre el objeto hilo

Creación de hilos.- ej

```
import java.io.*;
public class T extends Thread {
    protected int n;
    Public static void main(String[] argv) {
        for (int i = 0; i<3; i++) new T(i).start();
    }
    public T(int n) {this.n = n;}
    public void run() {
        int i = 0;
        while (count < 5) {
            System.out.println("Hilo " + n + " iteracion " + i++);
            try {sleep((n + 1) * 1000);}
            catch(InterruptedException e) {e.printStackTrace();}
        }
        System.out.println("El hilo " + n + " ha terminado");
    }
}
```

Estructura de un servidor

- Servidor secuencial

 - `while (msg = getMessage())`

 - Gestiona el mensaje

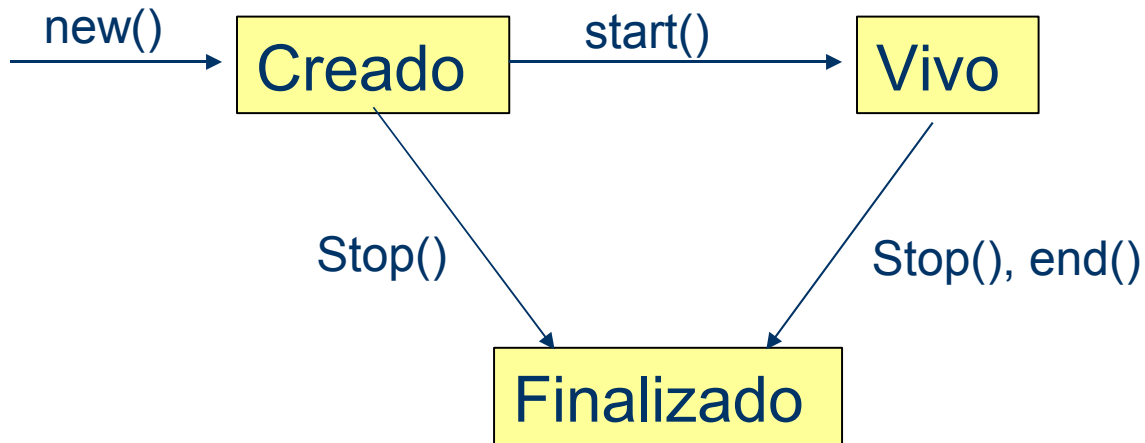
- Servidor concurrente

 - `while (msg = getMessage())`

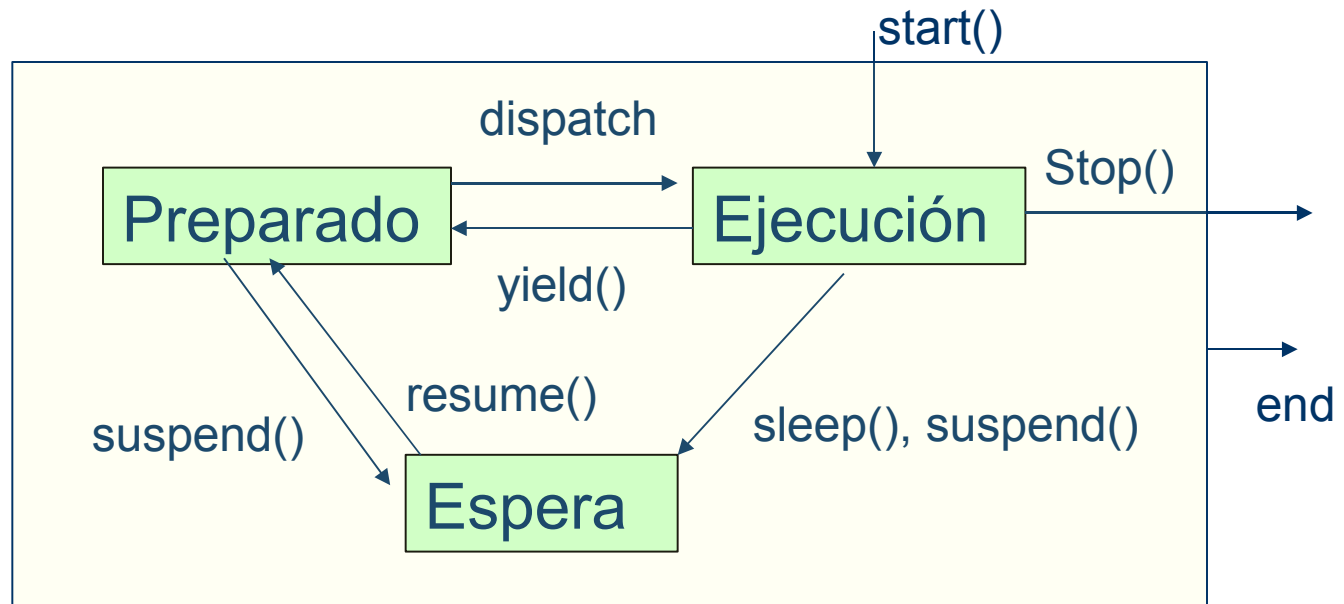
 - Crea nueva tarea para gestionar el mensaje

- El servidor concurrente puede atender varios clientes simultáneamente

Ciclo de vida de los hilos



Estados de un hilo vivo



Estados de un hilo vivo

- start
 - inicia la ejecución de 'run'
- isAlive
 - cierto si tarea iniciada y no finalizada
- stop
 - Finaliza la tarea
- suspend
 - Suspende temporalmente la tarea (hasta 'resume')
- sleep
 - Suspende la tarea un periodo especificado (en milisegundos)
- join
 - Suspende hasta que finaliza otra tarea
- interrupt
 - Aborta la espera iniciada con 'sleep', 'wait' o 'join'

Ejemplo de interferencias

- Suponemos un conjunto de cuentas bancarias
- Varios hilos utilizan esas cuentas
 - Cada hilo usa dos cuentas, transfiriendo n unidades de la primera a la segunda
 - La selección de la cuenta destino y la cantidad son aleatorias
 - Cada hilo cede el control (yield) en mitad de una transferencia
 - Simula una posible expulsión por parte del SO
 - Con ello aumentamos la probabilidad de interferencias
- La suma total (valor acumulado de todas las cuentas) debe ser constante

Ejemplo de interferencias

```
import java.io.*;
public class Interferencias {
    public static void main(String[] args) {
        Banco b = new Banco(10, 1000);
        for (int i = 0; i < numCuentas; i++)
            (new Transf(b, i, sadolnicial)).start();
    }
}
```

```
class Transf extends Thread {
    private Banco banco; private int from, max;
    private int rand(int n) {(int)n*Math.random();}
    public Transf (Banco b, int from, int max) {
        banco = b; This.from = from; this.max = max;
    }
    public void run() {
        try {
            while (!interrupted()) {
                banco.transfiere(from,
                    rand(banco.size()), rand(max));
                sleep(1);
            }
        } catch (InterruptedException e) {}
    }
}
```

```
class Banco {
    private int[] cuenta;
    private long ntransf = 0;
    public int size() {return cuenta.length; }
    public Banco(int n, int v0) {
        cuenta = new int[n]; ntransf = 0;
        for (int i = 0; i < size(); i++) cuenta[i] =
            v0;
    }
    public void transfiere(int from, int to, int n) {
        if (cuenta[from] < n) return;
        cuenta[from] -= n;
        Thread.currentThread().yield();
        cuenta[to] += n; if (++ntransf % 10
            == 0) test();
    }
    public void test() {
        int total = 0;
        for (int i = 0; i < size(); i++) total +=
            cuenta[i];
        System.out.println("Transferencias:" +
            ntransf +
            " Distribuidas bajo Internet");
    }
}
```

Sincronización y exclusión mútua

- Las tareas deben cooperar entre sí
 - Comparten objetos
 - Ej. una tarea modifica el estado del objeto, otra lo consulta
 - Pueden haber interferencias
 - Sólo pueden aparecer en fragmentos de código que acceden a objetos compartidos
 - Solución = garantizar ejecución secuencial de dichos fragmentos de código = exclusión mútua
 - Palabra reservada 'synchronized'
 - Puede aplicarse a métodos: *synchronized void m() {..}*
 - Y a bloques de código: *synchronized(objeto) {..}*
 - Se interpreta como una sección crítica que impide la ejecución simultánea de otros métodos sincronizados
 - La usamos para todo recurso que requiere acceso atómico

Ejemplo revisado

- En el ejemplo anterior teníamos problemas de interferencias en el acceso a las cuentas
- Usamos `synchronized` en las funciones que acceden a las cuentas (transferencia y test)
`public synchronized void transferencia(..) ..`
`public synchronized void test(..) ..`
- Con ello corregimos el problema
- Pero restringimos demasiado la concurrencia
 - Dos transferencias que trabajan sobre cuentas distintas no pueden interferir entre sí, pero las ejecutamos en exclusión mutua
 - La solución es restringir el acceso únicamente a las dos cuentas utilizadas por cada hilo

Espera y notificación

- Método parcial = método que sólo debe activarse en determinados estados del objeto
 - Ej.- extraer de una lista (sólo si no vacía)
- Métodos parciales en objetos compartidos
 - Si un hilo invoca un método parcial en un estado incorrecto, debe **esperar**
 - Cuando otro hilo modifica el estado debe notificar el cambio a un hipotético hilo en espera (**notificación**)
- Métodos de sincronización
 - wait.- suspende hilo, libera exclusión mútua
 - notify.- reactiva uno de los hilos suspendidos por wait
 - notifyAll.- idem., pero los reactiva todos

Ejemplo de espera y notificación

- Productor consumidor
 - Una hilo genera valores y los inserta en una cola
 - Otro hilo extrae valores de la cola y los escribe en pantalla
 - La cola mantiene orden FIFO (first-in first-out)
- Implementamos la cola como vector circular
 - Vector de enteros de talla N
 - Permite almacenar hasta N valores generados y todavía no consumidos
 - Tres variables que representan el índice donde insertar, índice donde extraer, y número de elementos (i,e,n)
 - Operaciones
 - void put(int x)
 - int get()
 - boolean lleno()
 - boolean vacio()

Ejemplo de espera y notificación

```
public class prodCons {
    public static void main(char[] args) {
        Buffer b= new Buffer(4);
        (new Productor(b,20)).start();
        (new Consumidor(b)).start();
    }
}

class Productor extends Thread {
    private int N;
    public Productor (int max) {N=max;}
    public void run() {
        for (int i=0; i<N; i++) b.put(i);
        b.put(-1); System.out.println("Fin del productor");
    }
}

class Consumidor extends Thread {
    public void run() {
        do{ int x=b.get(); System.out.println(" "+x); } while (x>=0);
        System.out.println("Fin del consumidor");
    }
}
```

```
class Buffer {
    private int[] v;
    private int N, n, i, e;
    private boolean lleno() {return n==N;}
    private boolean vacio() {return n==0;}

    public Buffer(int max) {
        N=max; n=e=i=0; v=new int[N];
    }
    public synchronized void put(int x) {
        while (lleno()) wait();
        v[i]=x; i=(i+1)%N; n++;
        notifyAll();
    }
    public synchronized int get() {
        while (vacio()) wait();
        int x=v[e]; e=(e+1)%N; n--;
        notifyAll();
        return x;
    }
}
```

Interbloqueos

- Interbloqueo = dos o más procesos se están esperando mutuamente
 - Los hilos utilizan recursos (hard o soft). Ej.- cuentas del banco
 - Para evitar interferencias los recursos se solicitan antes de su uso, y se liberan tras su uso
 - Si un hilo solicita un recurso asignado a otro hilo, debe esperar
 - Es posible que A espera un recurso asignado a B, mientras B espera un recurso asignado a A
- Nunca podrán salir de esa situación
- Ej
 - Dos procesos Unix establecen comunicación bidireccional con dos pipes
 - Cruce de calles con prioridad para 'derecha libre'
 - Puente estrecho
 - Banco cuando bloqueamos sólo las cuentas afectadas por la transferencia
 - pide(from); pide(to); transfiere(n); libera(from); libera(to);

Análisis del problema

- Condiciones de Coffman (necesarias para interbloqueo)
 - Exclusión mútua.- queremos compartir recursos que no se pueden usar de forma simultánea
 - Uso y espera.- los recursos se solicitan y obtienen progresivamente
 - No expulsión.- sólo puede liberar un recurso quien lo usa
 - Espera circular.- espera mútua (ciclo en el GAR)
- Grafo de asignación de recursos (GAR)
 - Representamos hilos y recursos
 - Si un hilo solicita un recurso, dibujamos un arco del hilo al recurso
 - Si un recurso está asignado a un hilo, dibujamos un arco del recurso al hilo

Solución

- Prevención
 - Consiste en ‘romper’ alguna de las condiciones de Coffman
 - En muchos casos costoso o imposible
 - La más fácil de romper suele ser la espera circular
 - Ej.- En el caso del banco siempre ordenamos las cuentas en orden creciente (pedimos primero la menor, y luego la mayor)
- Evitación.-
 - Antes de asignar un recurso solicitado, analizamos si puede conducir a interbloqueos (algoritmo del banquero)
- Detección
 - Detección de ciclos en el GAR